

# Purpose

1. **Purpose:** The purpose of this toolkit is to provide game developers with a customizable and easy-to-integrate settings menu UI for their projects.
2. **Simplification:** This toolkit simplifies the process of creating user-friendly settings menus, allowing developers to focus more on game mechanics and less on UI design.
3. **Supported Settings:** It supports various settings such as graphics, audio, controls, and more, making it adaptable for different types of games.

# Getting Started

1. **Prefab Integration:** Simply drag and drop the `URP` or `HDRP` prefab into your scene to integrate the settings menu into your game.
2. **Automatic Setup:** The settings menu will automatically integrate into your game's UI system without any additional setup required.
3. **Navigation:** Navigate to the `Customize` asset inside the `OneTapUI` folder.
4. **Customization Options:** Here, you can make all the customizations accessible without any coding or UI design.
5. **Compatibility Selection:** Select your `Input System` and `Render Pipeline` under **Other Customizations** to ensure the settings menu works seamlessly with your game's existing systems.
  - **Input System Options:**
    1. **Legacy Input System:** Built into Unity as the default input system for most projects. Although not recommended, it can still be used if necessary.
    2. **Input System Package:** Introduced in Unity 2019.1, this system is more powerful and flexible than the legacy system. Now comes with the default templates in Unity 6. Recommended for new projects due to its versatility.
  - **Render Pipeline Options:**
    1. **Universal Render Pipeline:** A lightweight, scriptable render pipeline optimized for performance and efficiency, recommended for most projects.
    2. **High Definition Render Pipeline:** Optimized for high-end platforms, this pipeline is suitable for projects requiring high-quality graphics.
6. **Ready to Go:** Now, you are ready to go! Test the settings menu in play mode to see how it looks and functions in your game.
7. **Customization Potential:** If you aren't satisfied with the default settings, you can further customize the settings menu in the next section to better suit your game's needs.

# Theme Maker

1. **Navigation to Customize:** Navigate to `Customize`.
2. **Tab Direction:** Select the tab direction to change the layout of the tabs.
  - **Horizontal Tabs:** Displays tabs horizontally at the top of the settings menu.
  - **Vertical Tabs:** Displays tabs vertically on the left side of the settings menu.
3. **Layout Options:** Choose from the preset layouts.
  - **Floating:** Displays the settings menu as a floating window in the center of the screen.
  - **Containered:** Similar to floating but with a semi-transparent background covering the entire screen.
  - **Outlined:** Displays the settings menu as a floating window with an outline around it.
  - **Transparent:** Features only a semi-transparent background covering the entire screen.

- **Half Transparent:** Transparent with solid tabs.
- **Half Outlined:** Semi-transparent with outlines.

## Other Customizations

1. **Skipped Section:** We've already covered `Input System` and `Render Pipeline` in the **Getting Started** section.
2. **Blur Value:** Adjusts the background blur of the settings menu. A higher value means more blur. `Default: 3`.
3. **Blur Duration:** The time it takes to blur the background. `Default: 0.5`.
4. **Tab Height:** Adjusts the height of the tabs. Changing this value is not recommended if using the default layouts. `Default: 50`.

## Menu Sound Effects

1. **Click Sound Effects:** Add or remove audio clips to be played in random order upon clicking.

## Menu Specific Settings

### Menu Names

1. **Summary:** These menus will display once the escape key is pressed.
2. **None State:** Defines the state where no menu is displayed. **DON'T DELETE IT!** It won't be shown anyway.
3. **Explained Functions:** Each member in this list is bound to a function. One is called on deinitialize, and the other on initialize, found in `Scripts > UI > MenuController`.
4. **Initialize Functions:** `OnInitialize` is called when the menu is displayed, and `OnDeinitialize` is called when the menu closes.
5. **Custom Functions:** You can add custom functions to the related switch-case functions.

## Tabs, Variables, and Options

1. **Tabs:** Add or remove tabs. Each tab has a name and a list of variables, displayed either on the top or left side of the settings menu.
2. **Variable Types:** Each variable has a name, main type, input type, and a list of options.
  - **Main Type Options:** Defines the main type of the variable (`Dropdown`, `Slider`, `Action Dropdown`, or `Binding`).
  - **Input Type Options:** Specifies the input type, either `Float` or `Int`. `Float` can only be used with `Slider` variables for now.
  - **Options List:** A list of options shown when the variable is clicked, used for `Dropdown` and `Action Dropdown` variables. Regular dropdowns allow manually typed option names, while action dropdowns generate options procedurally using functions.

## Functional Variables

### Display/Graphics Data

1. **Navigate to Graphics Data:** Go to `Scripts > UI > Data > {Tab}Data.cs`.
2. **Function Types:** There are two types of functions: `Set<Variable>` and `Get<Variable>`.
3. **Set Function Example:**

```
public List<Vector2Int> resolutions = new()
{
    new Vector2Int(1920, 1080), new Vector2Int(1600, 1000),
    new Vector2Int(1280, 720), new Vector2Int(1280, 800)
};

public void SetResolution(int level)
{
    var resolution = resolutions[level];
    Screen.SetResolution(resolution.x, resolution.y, Screen.fullScreen);
}
```

4. **Purpose of Set Function:** The `Set<Variable>` function is obligatory for processing settings customized by the player.

5. **Resolution Indexing:** Each resolution option corresponds to an index, referred to as `level` in the inputs section.

- 0 -> 1920x1080
- 1 -> 1600x1000
- 2 -> 1280x720
- 3 -> 1280x800

6. **Procedural Options:** Instead of taking string arrays as option parameters, options are generated procedurally using functions. Here's an example:

```
public void GetResolutionOptions(List<string> options)
{
    for (int i = 0, count = resolutions.Count; i < count; i++)
    {
        options.Add($"{resolutions[i].x}x{resolutions[i].y}");
    }
}
```

7. **List Reference:** Since lists are reference types, adding values to the `options` list is sufficient.

8. **Quality Presets:** They manage themselves. Simply write the names of the variables inside, and they will automatically set their values upon change. For example, if `Preset` is assigned to "very low", which is the first value, it will also set motion blur to 0 (disabled).

## SRP specific settings

1. **Why?:** The settings menu is designed to work with both URP and HDRP. However, some settings has different implementations on each render pipeline.
2. **Assembly Definition:** The settings menu uses assembly definitions to separate URP and HDRP specific settings. `OneTapUI > SRPs` folder contains the assembly definitions for each render pipeline. For more information on the topic, check the **Unity Documentation**.
3. **Defined Booleans:** We have defined `USING_URP` and `USING_HDRP` booleans in the assembly definitions. These booleans are used to determine which settings to use in the settings menu. They can be used in the compiler level so inexistant settings won't be compiled.
4. **Example:** Here is an example of how to use these booleans in the code:

```
#if USING_URP
// URP specific code
#elif USING_HDRP
// HDRP specific code
#endif
```

## Audio Data

1. **Tag List:** Here are tags for your audio components. You can add or remove from the list.
2. **Tag Assignment:** Assign the tags of objects that play related sounds.
3. **Audio Sources Button:** Navigate to the section with the `Get AudioSources` button. Clicking this will scan each object in your scene for audio sources.
4. **Adding on the Fly:** To add objects on the fly, call the add function in the `Volume Controller` found in the sample prefab.

PLOXLAB  
ONETAPUI

